

[CS230] Deep Neural Networks: MLP Architecture

Lecturer: Gemini (Integrated Editor)

□ Course Table of Contents

Chapter 1-3. Foundations & Shallow Networks - *Completed*

Chapter 4. Deep Neural Networks (Current Unit)

- 4.1 Deep L-Layer Neural Network Architecture
 - * General Notation ($L, n^{[l]}$)
 - * Hierarchical Representation (Why Deep?)
 - * Matrix Dimensions Analysis
 - * Building Blocks Implementation
- 4.2 Forward Propagation in Deep Network
- 4.3 Deep Network Backpropagation (Overview)

지난 시간 복습 및 연결

우리는 지금까지 은닉층이 하나뿐인 '얕은 신경망'을 다뤘습니다. 하지만 현실 세계의 복잡한 문제 (자율주행, 자연어 처리 등)를 풀기엔 뇌 용량이 부족합니다. 이제 우리는 은닉층을 2개, 3개, 아니 수백 개까지 쌓아 올릴 것입니다. 이것이 바로 여러분이 매일 듣는 '딥러닝(Deep Learning)'의 실체입니다. 단순히 층만 늘리는 게 아니라, 코드를 **일반화(Generalization)**하여 어떤 깊이의 모델도 만들 수 있는 건축가가 되어 봅시다.

1 Unit Overview

□ 핵심 목표

이 단원은 L 개의 층을 가진 일반화된 심층 신경망(Deep MLP)을 설계하고 구현합니다.

- **표기법:** 층의 개수가 L 개일 때의 파라미터($W^{[l]}, b^{[l]}$)와 활성화값($A^{[l]}$)을 정의합니다.
- **원리:** 딥러닝이 데이터를 계층적(Hierarchical)으로 이해하는 방식(점 → 선 → 면)을 배웁니다.
- **차원:** 각 층의 뉴런 개수($n^{[l]}$)만 보고도 가중치 행렬의 크기를 즉시 계산해냅니다.
- **구현:** 하드코딩($W1, W2...$)을 버리고, 'for-loop'와 'Dictionary'를 이용해 유연한 코드를 작성합니다.

2 Essential Terminology: The Deep Notation

얇은 신경망에서 쓰던 표기법을 확장합니다. l 은 현재 층 번호를 의미합니다.

기호	의미	설명
L	전체 층 수	입력층(0번)을 제외한 층의 개수.
$n^{[l]}$	l 번째 층의 뉴런 수	$n^{[0]} = n_x$ (입력), $n^{[L]}$ (출력).
$g^{[l]}$	l 번째 층의 활성화 함수	보통 은닉층은 ReLU, 출력층은 Sigmoid.
$A^{[l]}$	l 번째 층의 출력	$A^{[l]} = g^{[l]}(Z^{[l]})$. 다음 층의 입력이 됨.

3 Core Concepts: 왜 깊게 쌓는가?

3.1 1. Hierarchical Representation (계층적 표현)

”교수님, 그냥 은닉층 1개에 뉴런 100만 개를 넣는 게(Wide), 10만 개씩 10층 쌓는 것(Deep)보다 낫지 않나요?” 아닙니다. 딥러닝의 힘은 '쪼개서 이해하기'에서 나옵니다.

□ 사람의 얼굴 인식 과정 (직관적 비유)

우리의 뇌나 딥러닝 모델은 복잡한 이미지를 한 번에 이해하지 않습니다.

1. **Layer 1 (Low-level):** 픽셀을 보고 가로선, 세로선 같은 **경계(Edges)**를 찾습니다.
2. **Layer 2 (Mid-level):** 선들을 조합해서 눈, 코, 귀 같은 **부분(Parts)**을 만듭니다.
3. **Layer 3 (High-level):** 부분들을 조합해서 **사람 얼굴(Face)** 전체를 인식합니다.

층을 깊게 쌓으면, 적은 파라미터로도 매우 복잡한 함수(사람 얼굴 등)를 효율적으로 표현할 수 있습니다.

3.2 2. Matrix Dimensions (차원 분석)

이 부분은 구현과 디버깅의 핵심입니다. 무조건 암기해야 합니다.

l 번째 층의 가중치 $W^{[l]}$ 와 편향 $b^{[l]}$ 의 크기는 다음과 같습니다.

- $W^{[l]}$ **Shape:** $(n^{[l]}, n^{[l-1]}) \rightarrow$ (현재 층 뉴런 수, 이전 층 뉴런 수)
- $b^{[l]}$ **Shape:** $(n^{[l]}, 1)$
- $Z^{[l]}, A^{[l]}$ **Shape:** $(n^{[l]}, m) \rightarrow$ (현재 층 뉴런 수, 데이터 개수)

□ 차원 계산 퀴즈 (실전 시나리오 & 계산)

상황: 입력 특성 $n_x = 12288$ (이미지). Layer 1 뉴런: 20개. Layer 2 뉴런: 7개.

질문: $W^{[1]}$ 과 $W^{[2]}$ 의 크기는?

- $W^{[1]}$: $(n^{[1]}, n^{[0]}) = (20, 12288)$
- $W^{[2]}$: $(n^{[2]}, n^{[1]}) = (7, 20)$

4 Implementation: Building Deep Network

이제 L 개의 층을 가진 신경망을 만듭니다. 'W1', 'W2' 변수를 따로 만들지 않고 'parameters['W' + str(l)]' 형태로 관리하는 것이 핵심입니다.

```
1 import numpy as np
2
3 class DeepNN:
4     def __init__(self, layer_dims):
5         """
6         layer_dims: 각 층의 뉴런 수를 담은 리스트
7                     예: [12288, 20, 7, 5, 1] (4-Layer Network)
8         """
9         self.params = {}
10        self.L = len(layer_dims) - 1 # 입력층 제외한 층 수
11
12        for l in range(1, self.L + 1):
13            # He Initialization (ReLU 사용 시 필수!)
14            # 0.01 대신 np.sqrt(2 / 이전 층 뉴런 수)를 곱함
15            self.params['W' + str(l)] = np.random.randn(layer_dims[l], layer_dims[l-1]) * np.sqrt(2/layer_dims[l-1])
16            self.params['b' + str(l)] = np.zeros((layer_dims[l], 1))
17
18            # 차원 확인 습관화(!)
19            assert(self.params['W' + str(l)].shape == (layer_dims[l], layer_dims[l-1]))
20
21        def forward(self, X):
22            """
23            [Linear -> ReLU] * (L-1) -> [Linear -> Sigmoid] * 1
24            """
25            caches = []
26            A = X
27            L = self.L
28
29            # 1. 은닉층 (1 ~ L-1): ReLU
30            for l in range(1, L):
31                A_prev = A
32                W = self.params['W' + str(l)]
33                b = self.params['b' + str(l)]
34
35                # Linear
36                Z = np.dot(W, A_prev) + b
37                # Activation (ReLU)
38                A = np.maximum(0, Z)
39
40                # 역전파를 위해 저장 (W, b, A_prev, Z)
41                caches.append((A_prev, W, b, Z))
42
43            # 2. 출력층 (L): Sigmoid 이진(분류)
44            W = self.params['W' + str(L)]
45            b = self.params['b' + str(L)]
46
47            Z = np.dot(W, A) + b
48            AL = 1 / (1 + np.exp(-Z)) # Sigmoid
49            caches.append((A, W, b, Z))
50
51            return AL, caches
52
53        # --- 실행 예제 ---
54        if __name__ == "__main__":
55            # 입력[(3) -> 은닉(5) -> 은닉(3) -> 출력(1)] 구조
56            layers = [3, 5, 3, 1]
```

```

57 model = DeepNN(layers)
58
59 # 가상의 데이터 (3 features, 4 samples)
60 X = np.random.randn(3, 4)
61
62 AL, _ = model.forward(X)
63 print("Output Shape:", AL.shape) # (1, 4) 예상

```

Listing 1: L-Layer Deep Neural Network Initialization & Forward

5 FAQ & Pitfalls

파라미터 초기화: 0.01 vs He Initialization (오해 방지 가이드)

얕은 신경망에서는 '* 0.01'로 초기화해도 괜찮았습니다. 하지만 층이 깊어지면 ($L > 5$), 값이 계속 곱해지면서 신호가 사라지거나 폭발합니다 (Vanishing/Exploding Gradient). 따라서 ReLU를 쓸 때는 **He Initialization** (' $\text{np.sqrt}(2/n)$ ')을 쓰는 것이 딥러닝의 표준 (Standard)입니다.

Q. Cache 리스트는 왜 만드나요?

A. 순전파(Forward)가 끝나면 바로 역전파(Backward)를 해야 합니다. 역전파 수식을 보면 Z , A_{prev} , W 값이 필요합니다. 이미 계산한 값을 버리지 않고 'caches'에 저장해두면, 다시 계산할 필요 없이 효율적으로 역전파를 수행할 수 있습니다.

다음 단계 (Next Step)

이제 여러분은 어떤 깊이, 어떤 구조의 신경망도 만들 수 있는 설계 능력을 갖췄습니다. 'layers' 리스트에 숫자만 바꿔 넣으면 됩니다.

하지만 깊은 신경망을 학습시키는 것은 생각보다 까다롭습니다. 다음 시간에는 이 모델을 가지고 '고양이 vs 개' 이미지를 분류하는 실제 프로젝트를 수행하며, 학습 과정에서 발생하는 다양한 문제들을 해결해 보겠습니다.

□ 단원 요약 (Cheat Sheet)

1. **Deep Learning:** 은닉층을 여러 개 쌓아 계층적 특징 (Hierarchical Features)을 학습한다.
2. **Dimensions:** $W^{[l]}$ 의 크기는 $(n^{[l]}, n^{[l-1]})$ 이다. (현재 층, 이전 층)
3. **Implementation:** 'for-loop'를 사용하여 L 번 반복하는 일반화된 코드를 작성한다.
4. **Initialization:** 깊은 망에서는 **He Initialization**을 사용하여 학습 불안정을 막는다.